PRODUCT REQUIREMENTS DOCUMENT (PRD)

NIYAT — Premium Skincare E-Commerce Platform

Document Version: 1.0
Document Status: Development Specification
Product Type: Full-Stack B2C E-Commerce Web Application
Brand: NIYAT
Primary Market: India
Currency: Indian Rupee (INR)
Frontend Framework: Next.js App Router + TypeScript
Backend Architecture: Next.js server-side application
Database: MongoDB + Mongoose
Authentication: Firebase Authentication
Payment Gateway: Razorpay
Product Image Storage: Cloudinary
Development Platform: Antigravity AI

---

1. EXECUTIVE SUMMARY

1.1 Product Overview

NIYAT is a skincare and personal care brand offering cosmetic products, including whitening cream, face wash, sunscreen, body lotion, and facial serums.

The objective is to develop a professional, secure, scalable, and visually attractive e-commerce platform where customers can discover NIYAT products, learn about their ingredients and usage, purchase products online, complete secure payments, and track their orders.

The platform will also include a comprehensive administrative dashboard through which authorized administrators can manage products, product images, inventory, prices, orders, customers, coupons, promotional banners, content, and sales reports without modifying the source code.

The website should establish a premium brand identity and provide a smooth shopping experience across mobile phones, tablets, and desktop devices.

1.2 Product Vision

To establish NIYAT as a trusted skincare brand with a premium digital shopping experience, efficient order management, and a scalable e-commerce infrastructure that supports future product launches and business growth.

1.3 Primary Objectives

1. Establish an official online sales channel for NIYAT.
2. Showcase the complete skincare product range professionally.
3. Enable customers to purchase products securely.
4. Integrate online payments and configurable Cash on Delivery.
5. Provide customers with accounts, order history, and shipment tracking.
6. Allow administrators to manage the complete e-commerce operation.
7. Improve organic product discovery through SEO.
8. Support discount campaigns, promotional banners, and marketing initiatives.
9. Maintain accurate inventory, order, and payment records.
10. Build a scalable foundation for future business expansion.

1.4 Success Criteria

The initial release will be considered successful when:

- Customers can browse and purchase published products.
- Prices, discounts, shipping fees, and order totals are calculated securely on the server.
- Razorpay test payments can be completed and verified end-to-end.
- Orders are stored persistently in MongoDB.
- Duplicate payment events cannot create duplicate orders or inventory deductions.
- Administrators can manage products and orders through a protected dashboard.
- Product images can be uploaded and managed through Cloudinary.
- The website works on mobile, tablet, and desktop.
- Essential SEO metadata and product structured data are implemented.
- Automated tests cover the critical shopping and payment workflows.
- Production deployment documentation is complete.

---

2. PROJECT SCOPE

2.1 In Scope — Initial Release

The first release will include:

- Public storefront and homepage.
- Product catalog and category pages.
- Product details pages.
- Search, filtering, sorting, and pagination.
- Shopping cart and checkout.
- Customer registration and authentication.
- Guest checkout.
- Customer account dashboard.
- Razorpay payment integration.
- Configurable Cash on Delivery.
- Order confirmation and tracking.
- Product and inventory management.
- Admin dashboard.
- Order and customer management.
- Coupons and promotional banners.
- Cloudinary image management.
- Email notifications.
- Tax and shipping configuration.
- Printable invoices.
- Product reviews from eligible customers.
- SEO, analytics readiness, accessibility, and performance optimization.
- Secure deployment and environment configuration.

2.2 Out of Scope for the Initial Release

The following may be developed in future phases:

- Native Android and iOS applications.
- Multi-vendor marketplace functionality.
- Seller registration and seller dashboards.
- International payments and multi-currency checkout.
- Subscription-based recurring orders.
- Loyalty points and referral rewards.
- AI-based personalized skincare recommendations.
- Automated courier allocation across multiple shipping providers.
- Advanced warehouse and distributor management.
- Wholesale customer pricing and B2B ordering.
- Complex marketing automation and customer segmentation.
- Multilingual storefront, unless explicitly approved for launch.

The architecture should not prevent these features from being introduced later.

---

3. USERS AND USER ROLES

3.1 Guest Customer

A visitor who has not signed in.

Permissions:

- Browse published products.
- Search and filter products.
- View product information.
- Add products to a guest cart.
- Apply eligible coupons.
- Complete guest checkout.
- Make a payment.
- View order confirmation.
- Access an order using a secure order lookup or tracking link, where implemented.

Restrictions:

- Cannot access another customer's orders or personal information.
- Cannot manage products, prices, or inventory.
- Cannot submit a verified-purchase review without satisfying review eligibility requirements.

3.2 Registered Customer

A customer authenticated through Firebase Authentication.

Permissions:

- All guest customer capabilities.
- Manage their profile.
- Save shipping addresses.
- View order history.
- View individual order details.
- Track shipments.
- Submit cancellation requests when eligible.

- Submit product reviews for eligible purchases.
- Manage wishlist items.
- Update communication preferences.

3.3 Administrator

An authorized user with full operational permissions.

Permissions:

- View dashboard metrics.
- Create, update, publish, archive, and delete eligible products.
- Manage product images.
- Manage prices and stock.
- Manage orders, shipments, cancellations, and refunds.
- View customer records when necessary.
- Configure coupons and promotional campaigns.
- Manage banners and website content.
- Configure shipping, taxes, and COD.
- View sales reports and export authorized data.
- Manage eligible staff accounts and permissions.
- Review administrative audit logs.
- Update site settings.

Administrator privileges must be granted through a trusted server-side process, never through public registration.

3.4 Staff User

An optional limited-privilege role.

Permissions must be configurable, such as:

- View and process orders.
- Update shipment status.
- View stock.
- Record approved inventory adjustments.

Staff must not automatically receive permission to change payment credentials, create administrators, or access all customer data.

3.5 System and Payment Provider

External systems interact with the platform through authenticated APIs and verified webhooks.

Examples:

- Razorpay payment events.
- Firebase identity verification.
- Email delivery events.
- Optional shipping provider updates.

External callbacks must not be trusted until their authenticity has been verified.

---

4. BRAND IDENTITY AND DESIGN REQUIREMENTS

4.1 Visual Direction

The website must communicate premium skincare, product quality, elegance, and trust.

Suggested design palette:

- Deep emerald green: primary brand color.
- Sage green: supporting brand color.
- Warm ivory: primary background.
- White: product cards and content surfaces.
- Metallic gold: restrained accent.
- Dark charcoal: body text.

Exact color values may be finalized during the design phase.

4.2 Typography

Use a premium, readable typography system.

Requirements:

- Distinct headings and body typography.
- Clear pricing and product labels.
- Consistent line heights and spacing.
- Readable mobile text.
- Accessible contrast between text and backgrounds.

4.3 Product Photography

The provided NIYAT packaging photographs are the source of truth for the products they depict.

Requirements:

- Preserve the actual NIYAT brand name and packaging.
- Use high-resolution product photographs.
- Provide consistent image aspect ratios.
- Support multiple images per product.
- Include zoom or gallery functionality.
- Optimize image loading.
- Provide meaningful alt text.
- Avoid stretching or distorting packaging.
- Do not represent generic or AI-generated packaging as an authentic NIYAT product.

Products without approved photographs should use an unpublished placeholder until the administrator uploads genuine images.

4.4 User Interface Standards

Every page must include appropriate:

- Loading states.
- Empty states.
- Error states.
- Success feedback.
- Form validation.
- Responsive navigation.
- Keyboard accessibility.
- Focus indicators.
- Accessible dialogs and menus.
- Consistent button and input styling.

Animations must be subtle, optional where appropriate, and must not interfere with shopping or accessibility.

---

5. PRODUCT CATALOG REQUIREMENTS

5.1 Initial Products

The platform will support the following six products.

Product	Size	Category
Whitening Cream	40 g	Face Cream
Face Wash	100 g	Facial Cleanser
Sunscreen	50 g	Sun Protection
Body Lotion	200 g	Body Care
Emergency Whitening Serum	3 ml	Facial Serum
Premium Face Serum	15 ml	Facial Serum

These categories are initial organizational suggestions and should be editable by the administrator.

Product prices, SKU values, ingredients, and specifications must be supplied by NIYAT.

5.2 Product Data Fields

Every product must support:

- Unique database identifier.
- Product name.
- URL slug.
- SKU.
- Barcode, if available.
- Short description.
- Full description.
- Category.
- Product size or volume.
- Regular price.
- Optional sale price.
- Currency.
- Product images.
- Cloudinary asset identifiers.
- Ingredients.
- Directions for use.
- Warnings and precautions.
- Verified benefits.
- Suitable skin types, if supported by product information.

- Verified SPF and PA information, where applicable.
- Available stock quantity.
- Low-stock threshold.
- Publication status.
- Featured product status.
- Tags.
- SEO title.
- Meta description.
- Creation timestamp.
- Last-updated timestamp.

5.3 Product Publication States

Products must support:

- Draft.
- Published.
- Archived.

Only published and available-to-sell products may appear in the public catalog, subject to stock and sale rules.

A published product may remain visible when out of stock, but purchasing must be disabled unless a specific backorder policy is configured.

5.4 Product Pricing

The platform must:

- Store monetary values using a precision-safe representation, preferably integer paise.
- Display prices in INR.
- Support regular and sale prices.
- Validate sale prices against regular prices.
- Calculate all prices on the server.
- Maintain an order snapshot of the price at purchase time.
- Never alter historical order totals when a product price changes.

5.5 Product Search and Filtering

Customers should be able to:

- Search by product name.
- Search by category.

- Filter by price range.
- Filter by availability.
- Sort by price ascending or descending.
- Sort by newest.
- Sort by popularity when supported by real sales data.
- Clear filters.
- Navigate paginated results.

Search must be case-insensitive where appropriate and must not expose unpublished products.

5.6 Product Reviews

Requirements:

- Only eligible customers may submit verified-purchase reviews.
- Ratings range from 1 to 5.
- Reviews may contain text.
- Review moderation must be available to administrators.
- Approved reviews appear on the product page.
- Average ratings must be calculated from genuine approved reviews.
- No fabricated reviews, ratings, or testimonials may be displayed.

---

6. WEBSITE INFORMATION ARCHITECTURE

6.1 Public Pages

1. Home
2. Shop All
3. Product Category
4. Product Details
5. Search Results
6. Shopping Cart
7. Checkout
8. Payment Result
9. Order Confirmation
10. Track Order
11. About Us
12. Contact Us

13. Frequently Asked Questions
14. Privacy Policy
15. Terms and Conditions
16. Shipping Policy
17. Return and Refund Policy
18. Cancellation Policy
19. 404 Not Found
20. General Error Page

6.2 Customer Account Pages

1. Register
2. Login
3. Forgot Password
4. Profile
5. Address Book
6. Order History
7. Order Details
8. Shipment Tracking
9. Wishlist
10. Review Management, where applicable

6.3 Administration Pages

1. Admin Login / Access Verification
2. Dashboard
3. Product Management
4. Add Product
5. Edit Product
6. Category Management
7. Inventory Management
8. Inventory Movement History
9. Order Management
10. Order Details
11. Payment Records
12. Refund Management
13. Customer Management
14. Coupon Management
15. Banner Management
16. Homepage Content Management
17. Review Moderation
18. Shipping Configuration
19. Tax Configuration

20. Email and Notification Configuration
21. Staff and Role Management
22. Reports and Exports
23. Audit Logs
24. Site Settings

---

7. FUNCTIONAL REQUIREMENTS — STOREFRONT

7.1 Homepage

The homepage must include:

- Configurable announcement bar.
- Brand logo and navigation.
- Search, account, wishlist, and cart icons.
- Hero banner with authentic product photography.
- Primary call to action.
- Featured product collection.
- Bestselling products based on real order data.
- Category navigation.
- Brand introduction.
- Product education content.
- Promotional campaign sections.
- Genuine customer testimonials, when available.
- Frequently asked questions.
- Newsletter signup.
- Contact and social media links.
- Footer with legal policies.

Administrators must be able to update homepage banners, announcements, and featured products without changing source code.

If a section has no published content, the storefront must display an appropriate fallback or hide that section.

7.2 Navigation

The header must support:

- Home.
- Shop.
- Product categories.
- About Us.
- Contact.
- Search.
- Customer account.
- Wishlist.
- Cart.

On mobile devices, navigation must use an accessible menu with clear touch targets.

7.3 Shopping Cart

The cart must support:

- Adding products.
- Removing products.
- Updating quantities.
- Displaying prices and line totals.
- Showing discounts.
- Showing estimated shipping costs when available.
- Showing the estimated total.
- Applying and removing coupon codes.
- Updating totals after cart changes.
- Handling stock changes.
- Empty-cart guidance.
- Continuing shopping.
- Proceeding to checkout.

A guest cart may be persisted in local browser storage for convenience. The server must still validate every item, price, discount, and stock quantity at checkout.

The cart must not be the authoritative source of order pricing.

7.4 Wishlist

Registered customers may:

- Add products to a wishlist.
- Remove products.
- View saved products.
- Move products to the cart.

Wishlist contents must be associated with the authenticated customer and must not be accessible to other users.

7.5 Checkout

Checkout must collect:

Customer information

- Full name.
- Mobile number.
- Email address.

Shipping information

- Address line 1.
- Address line 2, optional.
- City.
- State.
- PIN code.
- Country.

Additional information

- Billing address, if different.
- Optional order notes.
- Acceptance of required purchase terms.
- Optional marketing consent, separate from essential transaction processing.

Checkout must:

- Validate required fields.
- Validate Indian PIN codes where applicable.
- Validate contact information.
- Confirm shipping availability.
- Recalculate prices on the server.
- Revalidate coupon eligibility.
- Revalidate inventory.
- Display all charges before payment.
- Prevent duplicate order submissions.
- Preserve safe checkout progress when possible.

Guest checkout must be supported.

---

8. PAYMENT GATEWAY REQUIREMENTS

8.1 Razorpay Integration

Razorpay is the proposed online payment provider.

The initial implementation must use test credentials. Live payments must remain disabled until the merchant account is approved and production configuration is completed.

8.2 Payment Methods

Subject to Razorpay account activation, customers may use supported methods such as:

- UPI.
- Debit cards.
- Credit cards.
- Net banking.
- Supported wallets or other enabled payment methods.

Cash on Delivery may be offered separately as a configurable offline payment method.

8.3 Payment Workflow

1. Customer proceeds to checkout.
2. Server validates customer information, cart items, stock, shipping, discounts, taxes, and final amount.
3. Server creates a pending order with a unique order number.
4. Server creates a Razorpay order for the calculated amount.
5. Frontend launches Razorpay Checkout.
6. Customer completes or cancels payment.
7. Frontend sends the payment identifiers and signature to the backend.
8. Backend verifies the signature using the official Razorpay SDK.
9. Backend updates payment and order state according to verified results.
10. Razorpay webhooks provide additional payment-event verification and reconciliation.
11. Customer sees an accurate result page.
12. Order confirmation is sent after the appropriate confirmation conditions are satisfied.

8.4 Payment Security

The application must:

- Keep the Razorpay secret key on the server.
- Never expose private payment credentials in frontend bundles.
- Verify payment signatures on the backend.
- Verify webhook signatures against the original raw request body.
- Use idempotency or equivalent duplicate-event protection.
- Maintain unique payment references.
- Handle repeated and out-of-order webhook events safely.
- Reconcile uncertain payment outcomes.
- Avoid deducting inventory more than once.
- Never trust a client-side success message as proof of payment.
- Never store card numbers or CVVs.
- Record payment attempts and outcomes without storing unnecessary sensitive data.

8.5 Payment Statuses

Support at least:

- Created.
- Pending.
- Authorized, where applicable.
- Captured / Paid.
- Failed.
- Cancelled.
- Refund Pending.
- Partially Refunded, where applicable.
- Refunded.

Payment status must be stored separately from order fulfilment status.

8.6 Failed Payments

When a payment fails:

- Show a clear message.
- Do not mark the order as paid.
- Allow a safe retry where supported.
- Prevent duplicate stock deductions.
- Reconcile delayed provider events before finalizing an uncertain outcome.
- Expire abandoned pending orders according to a configurable policy.

8.7 Refunds

Authorized administrators may initiate eligible refunds through an appropriate workflow.

Requirements:

- Check refund eligibility.
- Store the refund amount and provider reference.
- Support partial refunds if permitted.
- Track refund status.
- Avoid processing the same refund request twice.
- Record administrative actions.
- Update customer-facing status only when supported by verified provider information.

---

9. CASH ON DELIVERY

COD must be configurable through the admin panel.

Requirements:

- Enable or disable COD globally.
- Configure a minimum or maximum order value if needed.
- Restrict COD to supported delivery regions.
- Show COD availability during checkout.
- Store the payment method as COD.
- Track payment collection separately from order fulfilment.
- Support COD confirmation and cancellation policies.
- Record collected payment after authorized confirmation.

COD orders must not be marked as paid before payment is actually collected and recorded.

---

10. ORDER MANAGEMENT

10.1 Order Creation

Every order must contain:

- Unique order number.
- Customer identifier, when registered.
- Guest customer details, where applicable.
- Purchased product snapshots.

- SKU and quantity.
- Unit price and line totals.
- Discount information.
- Tax amount, if applicable.
- Shipping charge.
- Final payable amount.
- Payment method.
- Payment status.
- Fulfilment status.
- Shipping address snapshot.
- Billing address snapshot, when applicable.
- Order creation timestamp.
- Order status history.
- Customer notes, if provided.
- Shipment and tracking information, when available.

10.2 Order Statuses

The system must support the following fulfilment states:

- Pending Payment.
- Confirmed.
- Processing.
- Packed, optional.
- Shipped.
- Out for Delivery, optional.
- Delivered.
- Cancellation Requested.
- Cancelled.
- Return Requested, if returns are supported.
- Returned, if applicable.

Payment failure and refund statuses must remain distinct from fulfilment statuses.

10.3 Admin Order Operations

Administrators must be able to:

- Search by order number.
- Filter by order date.
- Filter by payment status.
- Filter by fulfilment status.
- Filter by payment method.

- View purchased items.
- View shipping information.
- Update eligible fulfilment statuses.
- Add shipment tracking details.
- Review payment references.
- Print invoices.
- Export authorized order records.
- Process eligible cancellations and refunds.

Every status change must be recorded with the acting administrator, timestamp, previous status, and new status.

10.4 Order Confirmation

After successful payment verification or valid COD order acceptance:

- Generate a confirmation page.
- Display the order number.
- Show purchased items and the final amount.
- Send a confirmation email when email delivery is configured.
- Show the expected next steps.

Do not show a successful paid order merely because the browser returned from the payment gateway.

---

11. SHIPPING AND DELIVERY

11.1 Shipping Configuration

Administrators must be able to configure:

- Shipping regions.
- Shipping fees.
- Free-shipping threshold.
- Optional weight-based or order-value-based rules.
- Estimated delivery text.
- COD availability by region.
- Order processing time.
- Shipping policy content.

Initial delivery coverage is configurable and must be confirmed by NIYAT.

11.2 Shipment Management

For each shipped order, the system may store:

- Courier name.
- Tracking number.
- Tracking URL.
- Shipment date.
- Estimated delivery date, when provided.
- Shipment status.
- Delivery confirmation timestamp.

Manual tracking updates must be supported in the initial release.

Automated courier integration can be added later.

11.3 Shipping Notifications

Customers should receive configured notifications for:

- Order confirmation.
- Shipment dispatch.
- Tracking updates when available.
- Delivery confirmation where supported.
- Cancellation and refund status changes.

Email is the initial recommended channel. SMS and WhatsApp notifications may be added after their provider configuration and consent requirements are addressed.

---

12. INVENTORY MANAGEMENT

12.1 Inventory Fields

Each product must support:

- Available quantity.
- Reserved quantity, if a reservation policy is implemented.
- Low-stock threshold.

- Stock status.
- Stock movement history.

12.2 Inventory Operations

Administrators may:

- Add stock.
- Reduce stock through approved adjustments.
- Review inventory history.
- View low-stock products.
- Correct inventory errors with a recorded reason.

12.3 Inventory Rules

The system must:

- Validate stock during checkout.
- Prevent sales beyond available stock.
- Use atomic database operations or equivalent concurrency-safe controls.
- Prevent duplicate stock deduction from repeated payment events.
- Restore stock for eligible cancellations according to the inventory policy.
- Record every stock movement.
- Preserve inventory audit history.
- Avoid deleting historical stock records.

The exact inventory reservation and release policy must be defined before production deployment.

---

13. ADMIN DASHBOARD

13.1 Dashboard Overview

The dashboard should display:

- Total orders.
- Revenue from verified payments.
- Orders awaiting processing.
- Pending payments.

- Completed deliveries.
- Customer count.
- Low-stock alerts.
- Product sales summaries.
- Recent orders.
- Sales over configurable time periods.

All figures must come from actual database records.

Revenue calculations must define how cancellations, refunds, discounts, taxes, and COD collection are treated. Pending or failed payments must not be counted as captured revenue.

13.2 Product Management

Administrators can:

- Create products.
- Edit products.
- Upload and reorder images.
- Manage descriptions and ingredients.
- Configure prices.
- Update stock.
- Assign categories.
- Set SEO metadata.
- Feature products.
- Publish, unpublish, or archive products.

The interface must clearly indicate missing required information.

13.3 Category Management

Administrators can:

- Create categories.
- Edit category names and descriptions.
- Set category slugs.
- Assign category images.
- Assign products.
- Configure category SEO metadata.
- Archive categories when appropriate.

A category cannot be removed in a way that breaks historical orders or leaves products without a valid category.

13.4 Coupon Management

Support:

- Fixed-amount discounts.
- Percentage discounts.
- Minimum order values.
- Maximum discount limits.
- Start and expiration dates.
- Usage limits.
- Per-customer usage limits.
- Eligible products and categories.
- Active and inactive states.

Coupons must be validated on the server and applied before final order creation.

13.5 Banner and Content Management

Administrators can manage:

- Homepage hero banners.
- Announcement bar.
- Featured collections.
- Campaign banners.
- About Us content.
- FAQs.
- Contact information.
- Footer links.
- Legal policy pages.

Changes must be reflected on the storefront without requiring a code deployment, subject to the application's caching and revalidation configuration.

13.6 Reports and Exports

Provide authorized reports for:

- Orders by date range.
- Sales by product.
- Sales by category.
- Payment method.
- Payment status.
- Order fulfilment status.

- Inventory levels.
- Low-stock items.
- Coupon usage.
- Refunds.
- Customer order history, where permitted.

Allow CSV export for appropriate reports.

Exports must be permission-controlled and must not expose unnecessary personal information.

---

14. CUSTOMER ACCOUNT AND AUTHENTICATION

14.1 Firebase Authentication

Support:

- Email and password registration.
- Login and logout.
- Password reset.
- Email verification where configured.
- Optional Google sign-in.

14.2 Backend Verification

The backend must verify Firebase ID tokens using Firebase Admin SDK.

Customer identity must come from the verified token, not from a user ID supplied in the request body.

14.3 Profile Management

Customers may update permitted fields such as:

- Name.
- Contact information.
- Saved addresses.
- Communication preferences.

Sensitive account operations must require appropriate authentication and verification.

14.4 Access Control

Every protected API must enforce authorization on the server.

A hidden button or restricted frontend route is not a substitute for server-side access control.

Customers must never be able to:

- Read another customer's orders.
- Change their role to administrator.
- Modify authoritative prices.
- Change payment status.
- Modify another user's addresses.
- Access restricted admin endpoints.

---

15. CLOUDINARY IMAGE MANAGEMENT

15.1 Image Upload Requirements

Use Cloudinary for:

- Product images.
- Category images.
- Homepage banners.
- Optional approved editorial images.

15.2 Upload Workflow

1. Administrator selects an image.
2. Backend verifies the administrator's permission.
3. Backend authorizes the upload or generates a secure upload signature.
4. Cloudinary receives the image.
5. The application stores the relevant asset identifiers and secure URL.
6. The admin dashboard displays a preview.

15.3 Security and Performance

Requirements:

- Validate allowed image formats.
- Enforce upload size limits.
- Restrict upload permissions.
- Keep Cloudinary secrets on the server.
- Use optimized delivery transformations.
- Support replacement and removal.
- Avoid deleting an image that is still referenced elsewhere.
- Handle upload failures gracefully.

---

16. EMAIL AND CUSTOMER COMMUNICATION

16.1 Email Provider

Use a transactional email service such as Resend or an equivalent provider.

The provider must be configurable through environment variables.

16.2 Email Templates

Implement templates for:

- Welcome email, if enabled.
- Order confirmation.
- Payment confirmation.
- Payment failure or retry guidance, when appropriate.
- Shipment dispatch.
- Delivery confirmation, when supported.
- Cancellation confirmation.
- Refund status updates.
- Password reset through Firebase's supported flow.

16.3 Email Requirements

Emails must:

- Use consistent NIYAT branding.
- Display accurate order information.
- Include support contact details once supplied.
- Avoid exposing payment secrets.
- Be responsive.
- Be logged with delivery status where supported.
- Avoid duplicate notifications from repeated events.

Newsletter marketing consent must be separate from necessary transactional communication.

---

17. DATABASE DESIGN

MongoDB with Mongoose will be used for the primary application database.

17.1 Product

Suggested fields:

- _id.
- name.
- slug.
- sku.
- barcode.
- shortDescription.
- description.
- categoryId.
- size.
- regularPricePaise.
- salePricePaise.
- currency.
- images.
- ingredients.
- directions.
- warnings.
- verifiedBenefits.
- specifications.
- stockQuantity.
- lowStockThreshold.
- status.
- isFeatured.
- tags.

- seo.
- createdAt.
- updatedAt.

17.2 Category

Suggested fields:

- _id.
- name.
- slug.
- description.
- image.
- status.
- seo.
- createdAt.
- updatedAt.

17.3 UserProfile

Suggested fields:

- _id.
- firebaseUid.
- name.
- email.
- phone.
- role.
- addresses.
- communicationPreferences.
- createdAt.
- updatedAt.

Authentication credentials remain managed by Firebase. Do not store user passwords in MongoDB.

17.4 Address

Suggested fields:

- _id.
- userId.
- recipientName.

- phone.
- addressLine1.
- addressLine2.
- city.
- state.
- postalCode.
- country.
- isDefault.
- createdAt.
- updatedAt.

17.5 Order

Suggested fields:

- _id.
- orderNumber.
- userId, nullable for guest orders.
- customerSnapshot.
- items.
- shippingAddressSnapshot.
- billingAddressSnapshot.
- subtotalPaise.
- discountPaise.
- shippingPaise.
- taxPaise.
- totalPaise.
- currency.
- couponSnapshot.
- paymentMethod.
- paymentStatus.
- fulfilmentStatus.
- shipmentDetails.
- statusHistory.
- createdAt.
- updatedAt.

Each item must preserve the product name, SKU, size, unit price, quantity, discount, and line total at purchase time.

17.6 Payment

Suggested fields:

- _id.
- orderId.
- provider.
- providerOrderId.
- providerPaymentId.
- amountPaise.
- currency.
- status.
- signatureVerificationStatus.
- webhookEventReferences.
- refundReferences.
- createdAt.
- updatedAt.

Unique constraints must prevent duplicate provider payment references where appropriate.

17.7 Coupon

Suggested fields:

- _id.
- code.
- discountType.
- discountValue.
- minimumOrderPaise.
- maximumDiscountPaise.
- startAt.
- expiresAt.
- usageLimit.
- perCustomerLimit.
- applicableProductIds.
- applicableCategoryIds.
- status.
- createdAt.
- updatedAt.

17.8 InventoryMovement

Suggested fields:

- _id.
- productId.
- movementType.

- quantityChange.
- reason.
- relatedOrderId.
- actorId.
- createdAt.

17.9 Review

Suggested fields:

- _id.
- productId.
- userId.
- orderId.
- rating.
- reviewText.
- verificationStatus.
- moderationStatus.
- createdAt.
- updatedAt.

17.10 Additional Collections

Create suitable models for:

- Banner.
- SiteSettings.
- NewsletterSubscription.
- AuditLog.
- Refund, if refund lifecycle complexity warrants a separate collection.
- WebhookEvent or payment-event processing records.
- Cart, if server-side cart persistence is required.

17.11 Database Integrity

Requirements:

- Unique product slug.
- Unique SKU when present.
- Unique order number.
- Appropriate indexes on customer, order date, status, and payment references.
- Pagination for large result sets.
- Validation for monetary values.

- Safe concurrency handling.
- Historical order preservation.
- Retention and privacy controls for personal data.

Use MongoDB transactions when supported by the deployment. Document infrastructure requirements and implement safe alternatives where appropriate.

---

18. TECHNICAL ARCHITECTURE

18.1 Frontend

Use:

- Next.js App Router.
- TypeScript.
- Tailwind CSS.
- shadcn/ui or equivalent accessible components.
- React Server Components where appropriate.
- Client Components only when interactivity requires them.

Use server rendering for public product pages where appropriate.

Use client-side interactivity for cart controls, menus, dialogs, and other interactive elements.

18.2 Backend

Use Next.js Route Handlers and server-side services for:

- Product APIs.
- Customer profile APIs.
- Order APIs.
- Checkout validation.
- Payment creation and verification.
- Webhook handling.
- Admin operations.
- Upload authorization.
- Reporting.
- Configuration management.

Keep business logic in reusable server-side service modules rather than placing complex logic directly in UI components.

18.3 Database Access

- Connect to MongoDB through a reusable server-only database module.
- Use Mongoose schemas and validation.
- Reuse connections appropriately in serverless environments.
- Never access database credentials from the browser.
- Use atomic updates or transactions for sensitive stock and order operations.

18.4 Suggested Project Structure

app/

- (storefront)/
    - page.tsx
    - shop/
    - products/[slug]/
    - categories/[slug]/
    - cart/
    - checkout/
    - about/
    - contact/
- account/
    - login/
    - register/
    - orders/
    - profile/
    - addresses/
- admin/
    - dashboard/
    - products/
    - categories/
    - orders/
    - inventory/
    - customers/
    - coupons/
    - banners/
    - reports/
    - settings/
- api/
    - products/

    - categories/
    - checkout/
    - orders/
    - payments/
    - webhooks/
    - admin/
    - uploads/

components/

- layout/
- navigation/
- products/
- cart/
- checkout/
- account/
- admin/
- forms/
- ui/

lib/

- auth/
- db/
- payments/
- cloudinary/
- email/
- validation/
- security/
- pricing/
- shipping/

models/

- Product.
- Category.
- Order.
- Payment.
- UserProfile.
- Coupon.
- InventoryMovement.
- Review.
- Banner.
- SiteSettings.
- AuditLog.

services/

- productService.
- orderService.
- checkoutService.
- paymentService.
- inventoryService.
- couponService.
- notificationService.
- reportService.

tests/

- unit/
- integration/
- e2e/

The final structure may vary if the architecture remains modular, maintainable, and secure.

---

19. API REQUIREMENTS

All API routes must return consistent response formats and appropriate HTTP status codes.

19.1 Public APIs

- GET /api/products
- GET /api/products/[slug]
- GET /api/categories
- GET /api/banners
- POST /api/newsletter

Public APIs must return only published content.

19.2 Customer APIs

- GET /api/me
- PATCH /api/me
- GET /api/me/addresses
- POST /api/me/addresses
- PATCH /api/me/addresses/[id]

- DELETE /api/me/addresses/[id]
- GET /api/me/orders
- GET /api/me/orders/[id]
- POST /api/orders/[id]/cancel-request
- POST /api/products/[id]/reviews

Every customer-specific endpoint must verify the authenticated identity and resource ownership.

19.3 Checkout and Payment APIs

- POST /api/checkout/validate
- POST /api/orders
- POST /api/payments/razorpay/create-order
- POST /api/payments/razorpay/verify
- POST /api/webhooks/razorpay

The server must recalculate all authoritative totals.

Webhook routes must verify authenticity and process repeated events safely.

19.4 Admin APIs

- GET /api/admin/dashboard
- GET /api/admin/products
- POST /api/admin/products
- GET /api/admin/products/[id]
- PATCH /api/admin/products/[id]
- DELETE /api/admin/products/[id]
- GET /api/admin/orders
- GET /api/admin/orders/[id]
- PATCH /api/admin/orders/[id]/status
- GET /api/admin/customers
- GET /api/admin/inventory
- POST /api/admin/inventory/adjustments
- GET /api/admin/reports
- POST /api/admin/coupons
- PATCH /api/admin/coupons/[id]
- POST /api/admin/banners
- PATCH /api/admin/banners/[id]
- GET /api/admin/settings
- PATCH /api/admin/settings

Add further endpoints as needed for category management, refunds, content, shipping, staff roles, and audit logs.

The endpoint list is a specification of required capabilities, not a requirement to use every route exactly as written.

---

20. SEO REQUIREMENTS

SEO is a core requirement because customers must be able to discover NIYAT products through search engines.

Implement:

- Server-rendered product pages.
- Unique page titles.
- Unique meta descriptions.
- Canonical URLs.
- Open Graph metadata.
- Product structured data.
- Organization or brand structured data where appropriate.
- XML sitemap.
- robots.txt.
- Breadcrumb navigation and structured data where appropriate.
- Descriptive image alt text.
- Clean product slugs.
- Internal links between related products and categories.
- Search engine indexing controls for unpublished content.
- Google Search Console readiness.
- Google Analytics integration as an optional, consent-aware configuration.

Product structured data must reflect actual product information, including price and availability. Ratings and reviews must never be fabricated.

Do not guarantee a particular Google ranking.

---

21. PERFORMANCE REQUIREMENTS

The application must:

- Use optimized image delivery.
- Avoid unnecessary JavaScript.

- Use server rendering where it improves public page delivery.
- Cache public content appropriately.
- Revalidate cached pages when products, prices, or inventory change.
- Avoid stale availability or pricing at checkout.
- Paginate product and order listings.
- Use appropriate database indexes.
- Minimize layout shifts.
- Lazy-load below-the-fold images.
- Avoid blocking the main thread with unnecessary animations.

Target Core Web Vitals should meet Google's recommended thresholds under representative production conditions.

Initial targets:

- LCP: 2.5 seconds or less.
- INP: 200 milliseconds or less.
- CLS: 0.1 or less.

Measure performance on representative mobile devices and realistic network conditions.

---

22. SECURITY AND PRIVACY

22.1 Application Security

Implement:

- Firebase ID token verification.
- Server-side role-based access control.
- Input validation.
- Rate limiting.
- Secure HTTP headers.
- Appropriate CORS and origin restrictions.
- Protection against unauthorized access.
- Secure environment variable handling.
- Safe error messages.
- Audit logging.
- Abuse prevention for public forms and checkout.

22.2 Payment Security

- Keep private payment credentials server-side.
- Verify payment signatures.
- Verify webhook signatures.
- Implement idempotent processing.
- Reconcile uncertain payment outcomes.
- Never accept frontend totals as authoritative.
- Avoid storing unnecessary sensitive financial information.

22.3 Data Privacy

- Collect only information required for the service.
- Publish a clear privacy policy.
- Protect customer contact and address information.
- Restrict customer data access to authorized roles.
- Avoid unnecessary sensitive information in logs.
- Define retention and deletion procedures.
- Provide appropriate consent and communication preference controls.
- Review applicable Indian privacy and consumer-protection requirements before launch.

22.4 Administrator Security

- Secure initial administrator provisioning.
- Enforce authorization on every administrative API.
- Maintain audit records.
- Apply least-privilege staff permissions.
- Protect sensitive settings.
- Never allow public registration to create privileged accounts.

---

23. LEGAL AND PRODUCT-CONTENT REQUIREMENTS

NIYAT must provide accurate product information before publishing products.

The website must not invent:

- Ingredient lists.
- SPF or PA ratings.
- Clinical test results.
- Certifications.

- Dermatologist endorsements.
- Guaranteed whitening results.
- Specific timelines for visible changes.
- Medical or therapeutic claims.
- Product approvals or compliance credentials.

The business must confirm the applicable product labeling, cosmetic claims, tax, invoicing, consumer-protection, and e-commerce obligations.

The website must support:

- Terms and Conditions.
- Privacy Policy.
- Shipping Policy.
- Return and Refund Policy.
- Cancellation Policy.
- Contact and grievance information where legally required.
- Applicable tax and invoice fields.

Legal copy must be reviewed and approved before production launch.

---

24. NOTIFICATIONS AND OBSERVABILITY

24.1 Application Logging

Maintain structured logs for:

- API errors.
- Payment attempts.
- Payment verification results.
- Webhook processing.
- Order creation.
- Inventory changes.
- Administrative actions.
- Email failures.

Logs must not contain passwords, authentication tokens, payment secrets, or unnecessary sensitive personal information.

24.2 Monitoring

Configure appropriate monitoring for:

- Application errors.
- API availability.
- Database connectivity.
- Payment verification failures.
- Webhook processing failures.
- Email delivery errors.
- Inventory consistency issues.

Critical payment and order failures must be diagnosable without relying exclusively on browser logs.

24.3 Backups

Establish a database backup and restoration plan.

Before launch:

- Configure automated backups supported by the chosen MongoDB hosting provider.
- Define retention periods.
- Restrict backup access.
- Document restoration steps.
- Test restoration before treating the system as production-ready.

---

25. ANALYTICS AND MARKETING

25.1 Analytics

Prepare for integration with an approved analytics provider.

Track relevant events such as:

- Product view.
- Product search.
- Add to cart.
- Remove from cart.
- Begin checkout.
- Payment initiation.
- Verified purchase.

- Coupon applied.
- Newsletter subscription.

Purchase events must not be counted as successful merely because the customer opens a success page.

Use deduplication and verified order information where applicable.

25.2 Marketing Features

Support:

- Homepage promotional banners.
- Discount codes.
- Featured products.
- Sale pricing.
- Newsletter signup.
- Product recommendations based on configured rules.
- Social media links.
- Campaign landing pages.

Future enhancements may include abandoned-cart reminders, loyalty programs, referral codes, and automated customer segmentation.

Marketing communications must respect applicable consent and unsubscribe requirements.

---

26. TESTING REQUIREMENTS

26.1 Unit Tests

Test:

- Product price calculations.
- Discount calculations.
- Tax calculations.
- Shipping calculations.
- Coupon eligibility.
- Product validation.
- Order status transitions.
- Inventory adjustment rules.

- Payment signature verification.

26.2 Integration Tests

Test:

- MongoDB connectivity.
- Product CRUD operations.
- Firebase token verification.
- Admin access restrictions.
- Checkout validation.
- Order creation.
- Razorpay order creation using test configuration.
- Payment verification.
- Webhook verification.
- Duplicate event handling.
- Inventory consistency.
- Cancellation and refund state transitions.

26.3 End-to-End Tests

Test the following user journeys:

1. Browse a product.
2. Add it to the cart.
3. Change the quantity.
4. Apply a valid coupon.
5. Enter shipping information.
6. Complete a Razorpay test payment.
7. Verify order confirmation.
8. View the order in the customer account.
9. Process the order through the admin dashboard.
10. Update tracking information.
11. Confirm stock and order totals.
12. Test failed and cancelled payments.
13. Test unauthorized admin access.
14. Test out-of-stock products.
15. Test duplicate payment callbacks and webhook events.

26.4 Browser and Device Testing

Test current supported versions of:

- Chrome.
- Edge.
- Firefox.
- Safari.

Test representative:

- Android phones.
- iPhones.
- Tablets.
- Desktop screens.

26.5 Release Quality Gate

Do not launch if critical checkout, payment verification, order integrity, authorization, or inventory tests fail.

---

27. DEPLOYMENT REQUIREMENTS

27.1 Hosting

Suggested deployment:

- Next.js application: Vercel or a suitable Next.js-compatible hosting platform.
- Database: managed MongoDB hosting.
- Authentication: Firebase.
- Images: Cloudinary.
- Payments: Razorpay.
- Email: configured transactional email provider.

The selected host must support the required server-side execution, payment webhooks, database connections, and security controls.

27.2 Environment Variables

Provide a documented `.env.example`.

Potential variables include:

- MONGODB_URI

- NEXT_PUBLIC_FIREBASE_API_KEY
- NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN
- NEXT_PUBLIC_FIREBASE_PROJECT_ID
- FIREBASE_ADMIN_CREDENTIALS or an appropriate server identity configuration
- CLOUDINARY_CLOUD_NAME
- CLOUDINARY_API_KEY
- CLOUDINARY_API_SECRET
- RAZORPAY_KEY_ID
- RAZORPAY_KEY_SECRET
- RAZORPAY_WEBHOOK_SECRET
- EMAIL_PROVIDER_API_KEY
- EMAIL_FROM_ADDRESS
- NEXT_PUBLIC_SITE_URL
- SITE_URL
- ADMIN_PROVISIONING_CONFIGURATION

Exact names may vary by implementation.

Private credentials must never be committed to source control or exposed through public frontend variables.

27.3 Deployment Environments

Maintain separate:

- Local development.
- Testing/staging.
- Production.

Use separate payment credentials and appropriate database configuration for each environment.

27.4 Production Checklist

Before launch:

- Configure the production domain and HTTPS.
- Configure environment variables.
- Verify Firebase authorized domains.
- Configure Razorpay production credentials and webhooks.
- Verify payment and refund flows.
- Configure Cloudinary uploads.
- Configure email sending and domain authentication.
- Configure database backups.

- Review access control.
- Verify legal policies and product content.
- Test mobile responsiveness.
- Validate SEO metadata and sitemap.
- Run end-to-end tests.
- Confirm monitoring and error reporting.
- Test order and database recovery procedures.

---

28. ACCEPTANCE CRITERIA

28.1 Storefront

- The homepage loads with approved NIYAT branding.
- Published products appear in the shop.
- Product details display accurate information.
- Search and filtering work correctly.
- Mobile navigation is usable.
- Unpublished products are not publicly accessible for purchase.

28.2 Cart and Checkout

- Customers can add, remove, and update cart items.
- The server validates prices, stock, discounts, shipping, and totals.
- Invalid coupons are rejected.
- Checkout cannot exceed available inventory.
- Guest and registered customer checkout both work.

28.3 Payments

- Razorpay test orders can be created.
- Valid signatures are verified.
- Invalid signatures are rejected.
- Webhooks are authenticated.
- Repeated events do not duplicate payment or stock operations.
- Failed payments are not marked as paid.
- Uncertain outcomes are reconciled safely.

28.4 Administration

- Only authorized administrators can access admin APIs.
- Products can be created, updated, published, and archived.
- Product images can be uploaded and replaced.
- Stock updates are recorded.
- Orders can be filtered and processed.
- Coupons and banners can be managed.
- Reports are based on real records.

28.5 Customer Accounts

- Firebase authentication works.
- Password reset works where configured.
- Customers can view only their own orders.
- Addresses can be managed securely.
- Eligible customers can track their orders and submit reviews.

28.6 Quality and Deployment

- The application builds successfully.
- Critical automated tests pass.
- No production secrets are exposed.
- Error states are handled gracefully.
- SEO metadata is implemented.
- Deployment and recovery documentation is available.

---

29. DEVELOPMENT PHASES

Phase 1 — Discovery and Foundation

- Confirm business details.
- Finalize product information.
- Establish repository and environment configuration.
- Set up Next.js and TypeScript.
- Set up Tailwind CSS and UI components.
- Configure MongoDB and Firebase.
- Establish the brand design system.

Deliverable: Running application foundation.

Phase 2 — Storefront

- Homepage.
- Navigation.
- Shop page.
- Category pages.
- Product detail pages.
- Search and filtering.
- Responsive design.
- SEO metadata.

Deliverable: Functional public storefront.

Phase 3 — Customer Shopping

- Cart.
- Wishlist.
- Authentication.
- Customer profiles.
- Address book.
- Checkout forms.
- Price and stock validation.

Deliverable: End-to-end shopping flow up to order creation.

Phase 4 — Payments and Orders

- Razorpay test integration.
- Signature verification.
- Webhook handling.
- Payment state management.
- Order confirmation.
- Email notifications.
- COD configuration.

Deliverable: Verified test-payment and order lifecycle.

Phase 5 — Admin Dashboard

- Dashboard metrics.
- Product management.
- Category management.
- Image uploads.

- Inventory management.
- Order management.
- Coupon management.
- Banner and content management.
- Reports and exports.

Deliverable: Functional business administration system.

Phase 6 — Quality and Security

- Automated tests.
- Authorization review.
- Payment security review.
- Inventory concurrency tests.
- Performance optimization.
- Accessibility testing.
- SEO validation.

Deliverable: Release candidate.

Phase 7 — Production Launch

- Production hosting.
- Domain and HTTPS.
- Live payment configuration.
- Email configuration.
- Database backups.
- Monitoring.
- Product and policy verification.
- Final acceptance testing.

Deliverable: Production-ready NIYAT e-commerce website.

---

30. RISKS AND MITIGATION

Risk: Incorrect product claims

Mitigation: Require verified product information and administrative review before publishing.

Risk: Payment fraud or duplicate callbacks

Mitigation: Server-side signature verification, authenticated webhooks, idempotent processing, and payment reconciliation.

Risk: Incorrect inventory during simultaneous purchases

Mitigation: Atomic stock updates or transactions, reservation policies where necessary, and concurrency testing.

Risk: Unauthorized administrative access

Mitigation: Server-side role checks, secure administrator provisioning, and audit logs.

Risk: Incomplete product imagery

Mitigation: Keep products unpublished until approved images are available.

Risk: SEO pages showing stale pricing

Mitigation: Revalidate cached content and always recalculate authoritative prices during checkout.

Risk: Unconfigured business policies

Mitigation: Use configurable settings and block production launch until shipping, returns, taxes, and contact information are approved.

Risk: Hosting incompatibility

Mitigation: Confirm the deployment platform supports required server execution, webhooks, and database connectivity.

---

31. OPEN BUSINESS DECISIONS

The following decisions must be confirmed by NIYAT before final production configuration.

Decision	Status
Actual selling prices for six products	Pending
Approved product photographs	Partially available
Verified ingredients and product specifications	Pending

Official logo and brand assets	To confirm
Business email and telephone number	Pending
Official business address and applicable registration details	Pending
Shipping coverage	To confirm
Shipping fees and free-shipping threshold	To confirm
COD availability and restrictions	To confirm
Return, refund, and cancellation policies	To approve
Tax and invoice configuration	To confirm
Razorpay merchant onboarding	To complete
Transactional email provider	To select
Domain and hosting account	To confirm
Admin users and permissions	To configure
Analytics and advertising integrations	Optional
WhatsApp customer communication	Optional

Unconfirmed information must remain configurable and must not be replaced with fabricated details.

---

32. FINAL PRODUCT DEFINITION

NIYAT will be a full-stack, mobile-responsive skincare e-commerce platform built with Next.js App Router and TypeScript.

The system will provide a premium public storefront, six initial product listings, secure customer authentication, shopping cart, checkout, Razorpay integration, order tracking, inventory control, a protected administrative dashboard, Cloudinary image management, customer communication, reporting, and search-engine optimization.

MongoDB will store products, customers' application profiles, orders, payments, coupons, and operational records. Firebase Authentication will manage customer identity. Razorpay will process online payments. Cloudinary will manage product and promotional images.

All sensitive operations—including price calculations, authorization, inventory updates, payment verification, and order status transitions—must be enforced on the server.

The final deliverable is not merely a website design or prototype. It is a tested, documented, configurable e-commerce application prepared for production deployment once the required business details, external credentials, product information, and operational policies have been approved.

End of PRD — NIYAT E-Commerce Platform, Version 1.0.